Name: Bonita, Kier Nathaniel M.	Date Performed: 02/12/2026
Course/Section: CPE 212-CPE31S1	Date Submitted: 02/12/2026
Instructor: Engr. Robin Valenzuela	Semester and SY: 2nd - 2025-2026

Activity 6: Targeting Specific Nodes and Managing Services

1. **Objectives:**

1.1 Individualize hosts
1.2 Apply tags in selecting plays to run
1.3 Managing Services from remote servers using playbooks
1.4 Managing Ansible Roles for Modularization

2. **Intended Learning Outcomes:**

- Differentiate and organize remote hosts into appropriate inventory groups based on their server roles (e.g., web, database, file servers).
- Analyze playbook execution results and interpret changes made across different remote servers.
- Configure and manage system services remotely using Ansible modules (e.g., service module for starting and enabling services).
- Design and apply Ansible Roles to modularize configurations and improve playbook reusability, scalability, and maintainability.

3. **Discussion**:

In this activity, we try to individualize hosts. For example, we don't want apache on all our servers, or maybe only one of our servers is a web server, or maybe we have different servers like database or file servers running different things on different categories of servers and that is what we are going to take a look at in this activity.

We also try to manage services that do not automatically run using the automations in playbook. For example, when we install web servers or httpd for CentOS, we notice that the service did not start automatically.

Requirement:

In this activity, you will need to create another Ubuntu VM and name it Server 3. Likewise, you need to activate the second adapter to a host-only adapter after the installations. Take note of the IP address of the Server 3. Make sure to use the command *ssh-copy-id* to copy the public key to Server 3. Verify if you can successfully SSH to Server 3.

Task 1: Targeting Specific Nodes

1. Create a new playbook and named it site.yml. Follow the commands as shown in the image below. Make sure to save the file and exit.

```
---

- hosts: all
  become: true
  tasks:

  - name: install apache and php for Ubuntu servers
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

   - name: install apache and php for CentOS servers
     dnf:
       name:
         - httpd
         - php
       state: latest
     when: ansible_distribution == "CentOS"
```

Welcome | site.yml | inventory.ini

site.yml > { } 0 > [ ] tasks > { } 1 > when

Ansible Playbook - Ansible playbook files (ansible.json)

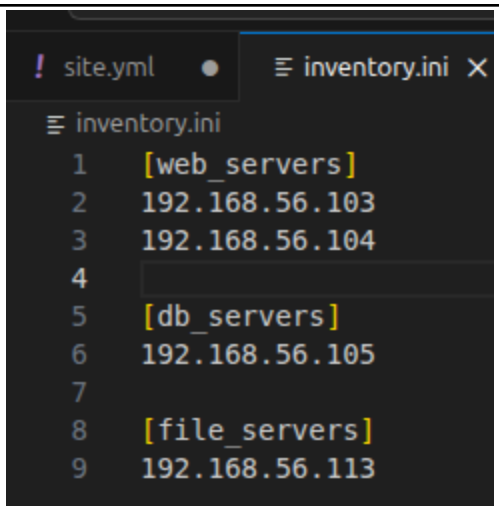
```yaml
---
- hosts: all
  become: true
  tasks:

    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache and php for CentOS servers
      dnf:
        name:
          - httpd
          - php
        state: latest
      when: ansible_distribution == "CentOS"
```

2. Edit the inventory file. Remove the variables we put in our last activity and group according to the image shown below:

```ini
[web_servers]
192.168.56.120
192.168.56.121

[db_servers]
192.168.56.122


[file_servers]
192.168.56.123
```

Make sure to save the file and exit.

Right now, we have created groups in our inventory file and put each server in its own group. In other cases, you can have a server be a member of multiple groups, for example you have a test server that is also a web server.



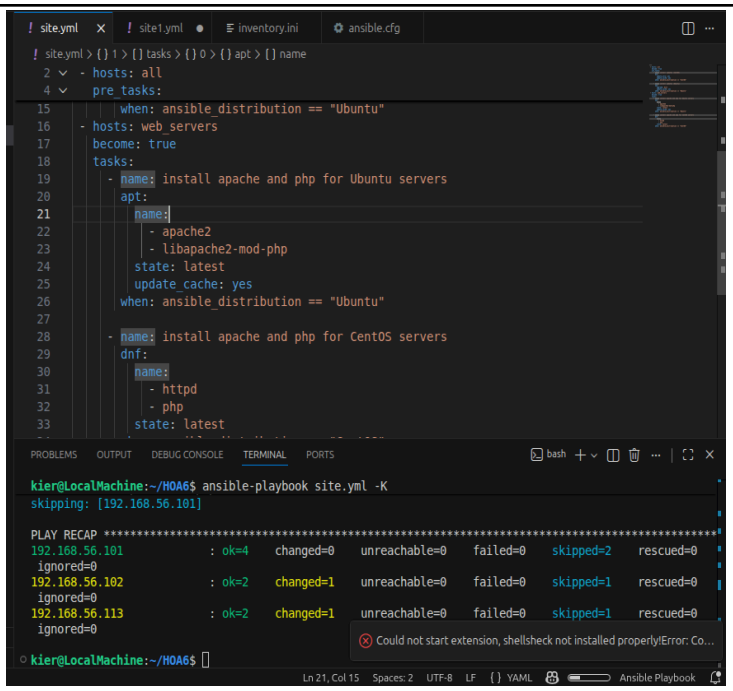
3. Edit the *site.yml* by following the image below:

```
---

- hosts: all
  become: true
  pre_tasks:

  - name: install updates (CentOS)
    dnf:
      update_only: yes
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install updates (Ubuntu)
    apt:
      upgrade: dist
      update_cache: yes
    when: ansible_distribution == "Ubuntu"


- hosts: web_servers
  become: true
  tasks:

  - name: install apache and php for Ubuntu servers
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
    when: ansible_distribution == "Ubuntu"

  - name: install apache and php for CentOS servers
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"
```

Make sure to save the file and exit.

The *pre-tasks* command tells the ansible to run it before any other thing. In the *pre-tasks*, CentOS will install updates while Ubuntu will upgrade its distribution package. This will run before running the second play, which is targeted at *web_servers*. In the second play, apache and php will be installed on both Ubuntu servers and CentOS servers.

Run the *site.yml* file and describe the result.

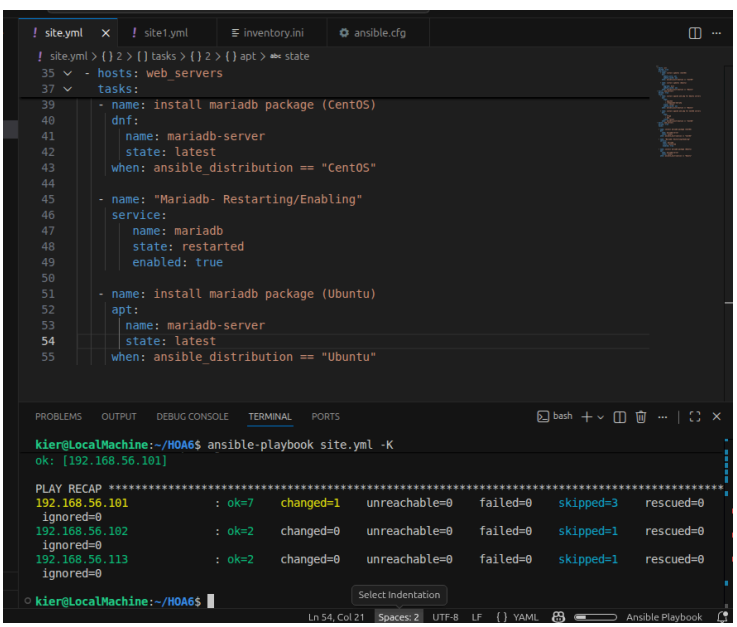


- All the servers should be updated depending on if they are using Ubuntu or CentOS. The web servers should also have Apache and PHP installed, but the package names are a little different based on the operating system.

4. Let's try to edit again the *site.yml* file. This time, we are going to add plays targeting the other servers. This time we target the *db_servers* by adding it on the current *site.yml*. Below is an example: (Note add this at the end of the playbooks from task 1.3.

```yaml
- hosts: db_servers
  become: true
  tasks:

  - name: install mariadb package (CentOS)
    yum:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: "Mariadb- Restarting/Enabling"
    service:
      name: mariadb
      state: restarted
      enabled: true

  - name: install mariadb packege (Ubuntu)
    apt:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "Ubuntu"
```

Make sure to save the file and exit.



Run the *site.yml* file and describe the result.

- **All the servers get a system updates based on whether they are using Ubuntu or CentOS. The web servers will also have Apache, PHP, and MariaDB installed, and MariaDB will be restarted and enabled so it starts automatically.**

5. Go to the remote server (Ubuntu) terminal that belongs to the db_servers group and check the status for mariadb installation using the command: *systemctl status mariadb.* Do this on the CentOS server also.

```
kier@server2: ~/Desktop
Warning: The unit file, source configuration file or drop-ins of mariadb.servic>
● mariadb.service - MariaDB 10.11.13 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: >
     Active: active (running) since Thu 2026-02-12 17:51:00 PST; 35min ago
       Docs: man:mariadbd(8)
             https://mariadb.com/kb/en/library/systemd/
   Main PID: 1253 (mariadbd)
     Status: "Taking your SQL requests now..."
      Tasks: 11 (limit: 76238)
     Memory: 109.5M (peak: 112.9M)
        CPU: 1.123s
     CGroup: /system.slice/mariadb.service
             └─1253 /usr/sbin/mariadbd

Feb 12 17:50:59 server2 mariadbd[1253]: 2026-02-12 17:50:59 0 [Note] InnoDB: Lo>
Feb 12 17:50:59 server2 mariadbd[1253]: 2026-02-12 17:50:59 0 [Warning] You nee>
Feb 12 17:50:59 server2 mariadbd[1253]: 2026-02-12 17:50:59 0 [Note] InnoDB: Bu>
Feb 12 17:50:59 server2 mariadbd[1253]: 2026-02-12 17:50:59 0 [Note] Server soc>
Feb 12 17:51:00 server2 mariadbd[1253]: 2026-02-12 17:51:00 0 [Note] /usr/sbin/>
Feb 12 17:51:00 server2 mariadbd[1253]: Version: '10.11.13-MariaDB-0ubuntu0.24.>
Feb 12 17:51:00 server2 systemd[1]: Started mariadb.service - MariaDB 10.11.13 >
Feb 12 17:51:00 server2 /etc/mysql/debian-start[1390]: Upgrading MariaDB tables>
Feb 12 17:51:00 server2 /etc/mysql/debian-start[1447]: Checking for insecure ro>
lines 1-23
```

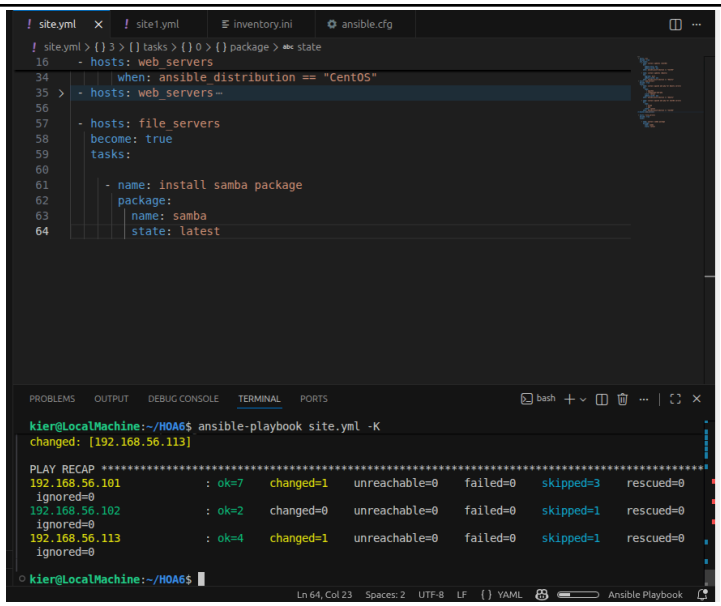
Describe the output.

- **This command shows the current status of Mariadb**

6. Edit the *site.yml* again. This time we will append the code to configure installation on the *file_servers* group. We can add the following on our file.

```
- hosts: file_servers
  become: true
  tasks:

  - name: install samba package
    package:
      name: samba
      state: latest
```

Make sure to save the file and exit.



Run the *site.yml* file and describe the result.

- **It installed samba package on the ip address that was in the [file_servers]**

The testing of the *file_servers* is beyond the scope of this activity, and as well as our topics and objectives. However, in this activity we were able to show that we can target hosts or servers using grouping in ansible playbooks.

Task 2: Using Tags in running playbooks

In this task, our goal is to add metadata to our plays so that we can only run the plays that we want to run, and not all the plays in our playbook.

1. Edit the *site.yml* file. Add tags to the playbook. After the name, we can place the tags: *name_of_tag*. This is an arbitrary command, which means you can use any name for a tag.

```yaml
---

- hosts: all
  become: true
  pre_tasks:

  - name: install updates (CentOS)
    tags: always
    dnf:
      update_only: yes
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install updates (Ubuntu)
    tags: always
    apt:
      upgrade: dist
      update_cache: yes
    when: ansible_distribution == "Ubuntu"
```

```yaml
- hosts: web_servers
  become: true
  tasks:

  - name: install apache and php for Ubuntu servers
    tags: apache,apache2,ubuntu
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
    when: ansible_distribution == "Ubuntu"

  - name: install apache and php for CentOS servers
    tags: apache,centos,httpd
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"
```

```
- hosts: db_servers
  become: true
  tasks:

  - name: install mariadb package (CentOS)
    tags: centos, db,mariadb
    dnf:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: "Mariadb- Restarting/Enabling"
    service:
      name: mariadb
      state: restarted
      enabled: true

  - name: install mariadb packege (Ubuntu)
    tags: db, mariadb,ubuntu
    apt:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "Ubuntu"

- hosts: file_servers
  become: true
  tasks:

  - name: install samba package
    tags: samba
    package:
      name: samba
      state: latest
```

Make sure to save the file and exit.
Run the *site.yml* file and describe the result.

```
site.yml   site1.yml   inventory.ini   ansible.cfg
site.yml > { } 3 > [ ] tasks > { } 0 > { } package > abc state
39    - hosts: web_servers
41      tasks:

56      - name: install mariadb package (Ubuntu)
57        tags: ubuntu, db, mariadb
58        apt:
59          name: mariadb-server
60          state: latest
61        when: ansible_distribution == "Ubuntu"
62
63    - hosts: file_servers
64      become: true
65      tasks:
66
67        - name: install samba package
68          tags: samba
69          package:
70            name: samba
71            state: latest
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
kier@LocalMachine:~/HOA6$ ansible-playbook site.yml -K
ok: [192.168.56.113]

PLAY RECAP *********************************************************************************************
192.168.56.101             : ok=7    changed=1    unreachable=0    failed=0    skipped=3    rescued=0
 ignored=0
192.168.56.102             : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0
 ignored=0
192.168.56.113             : ok=4    changed=0    unreachable=0    failed=0    skipped=1    rescued=0
 ignored=0
```

2. On the local machine, try to issue the following commands and describe each result:

2.1 *ansible-playbook --list-tags site.yml*

```
kier@LocalMachine:~/HOA6$ ansible-playbook --list-tags site.yml
      TASK TAGS: [always]

  play #2 (web_servers): web_servers    TAGS: []
      TASK TAGS: [apache, apache2, httpd, ubuntu]

  play #3 (web_servers): web_servers    TAGS: []
      TASK TAGS: [centos, db, mariadb, ubuntu]

  play #4 (file_servers): file_servers  TAGS: []
      TASK TAGS: [samba]
```

-

2.2 *ansible-playbook --tags centos --ask-become-pass site.yml*

```
kier@LocalMachine:~/HOA6$ ansible-playbook --tags centos site.yml -K
192.168.56.101             : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0
 ignored=0
192.168.56.102             : ok=3    changed=0    unreachable=0    failed=0    skipped=2    rescued=0
 ignored=0
192.168.56.113             : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0
 ignored=0
```

-

- ***it runs all commands that has the tag centos***

2.3 *ansible-playbook --tags db --ask-become-pass site.yml*

```
kier@LocalMachine:~/HOA6$ ansible-playbook --tags db site.yml -K
192.168.56.101             : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0
 ignored=0
192.168.56.102             : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0
 ignored=0
192.168.56.113             : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0
 ignored=0
```

-

- ***it runs all commands that has the tag db***

2.4 *ansible-playbook --tags apache --ask-become-pass site.yml*

```
kier@LocalMachine:~/HOA6$ ansible-playbook --tags apache site.yml -K
192.168.56.101             : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0
 ignored=0
192.168.56.102             : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0
 ignored=0
192.168.56.113             : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0
 ignored=0
```

-
- ***it runs all commands that has the tag apache***

2.5 *ansible-playbook --tags “apache,db” --ask-become-pass site.yml*

```
kier@LocalMachine:~/HOA6$ ansible-playbook --tags "apache,db" site.yml -K
192.168.56.101             : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0
 ignored=0
192.168.56.102             : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0
 ignored=0
192.168.56.113             : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0
 ignored=0
```

-
- ***it runs all commands that has the tag apache, db***

Task 3: Managing Services

1. Edit the file site.yml and add a play that will automatically start the httpd on CentOS server.

```yml
- name: install apache and php for CentOS servers
  tags: apache,centos,httpd
  dnf:
    name:
      - httpd
      - php
    state: latest
  when: ansible_distribution == "CentOS"

- name: start httpd (CentOS)
  tags: apache, centos,httpd
  service:
    name: httpd
    state: started
  when: ansible_distribution == "CentOS"
```

Figure 3.1.1

Make sure to save the file and exit.

You would also notice from our previous activity that we already created a module that runs a service.

```
- hosts: db_servers
  become: true
  tasks:

  - name: install mariadb package (CentOS)
    tags: centos, db,mariadb
    dnf:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: "Mariadb- Restarting/Enabling"
    service:
      name: mariadb
      state: restarted
      enabled: true
```

Figure 3.1.2

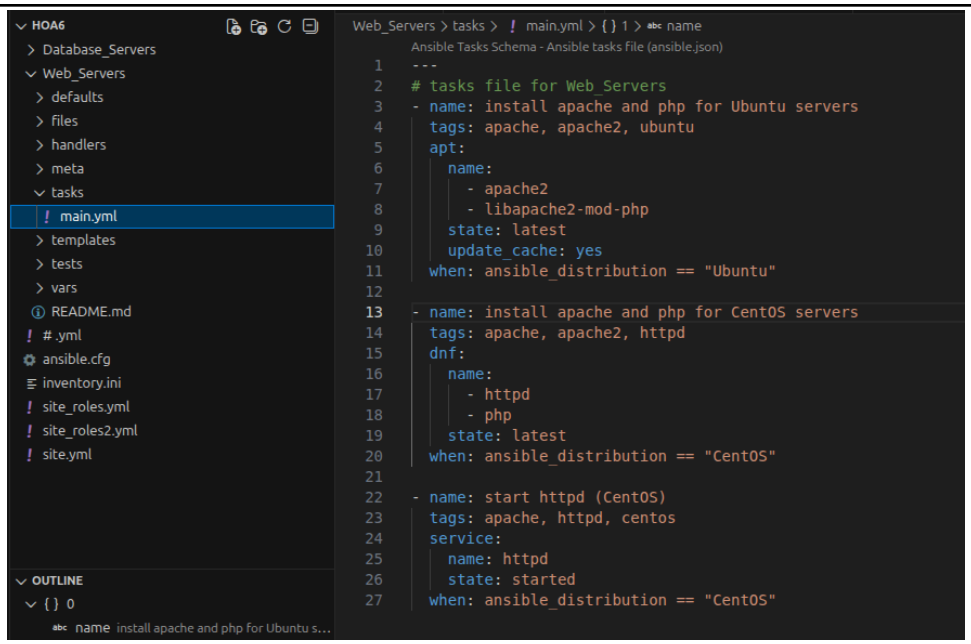
This is because in CentOS, installed packages' services are not run automatically. Thus, we need to create the module to run it automatically.

2. To test it, before you run the saved playbook, go to the CentOS server and stop the currently running httpd using the command *sudo systemctl stop httpd.* When prompted, enter the sudo password. After that, open the browser and enter the CentOS server's IP address. You should not be getting a display because we stopped the httpd service already.
3. Go to the local machine and this time, run the *site.yml* file. Then after running the file, go again to the CentOS server and enter its IP address on the browser. Describe the result.
   To automatically enable the service every time we run the playbook, use the command *enabled: true* similar to Figure 7.1.2 and save the playbook.

Supplementary Activity:

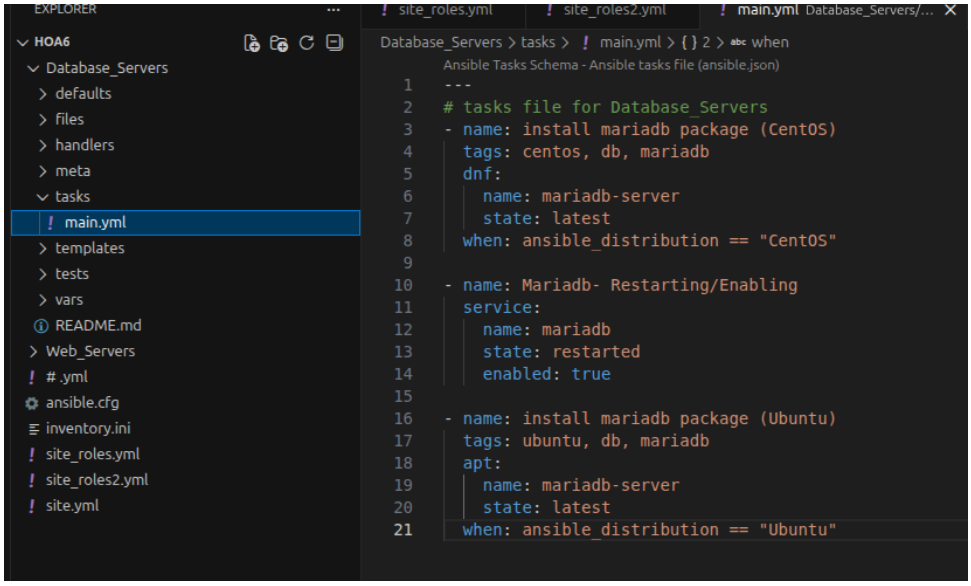
1. **Creating a Role for Web Servers**

-



2. Creating a Role for Database Servers

-



3. Combining Multiple Roles in a Playbook

site_roles.yml | site_roles2.yml | main.yml Database_Servers/... | # .yml | main.yml We

site_roles.yml > { } 0 > [ ] roles > abc 0

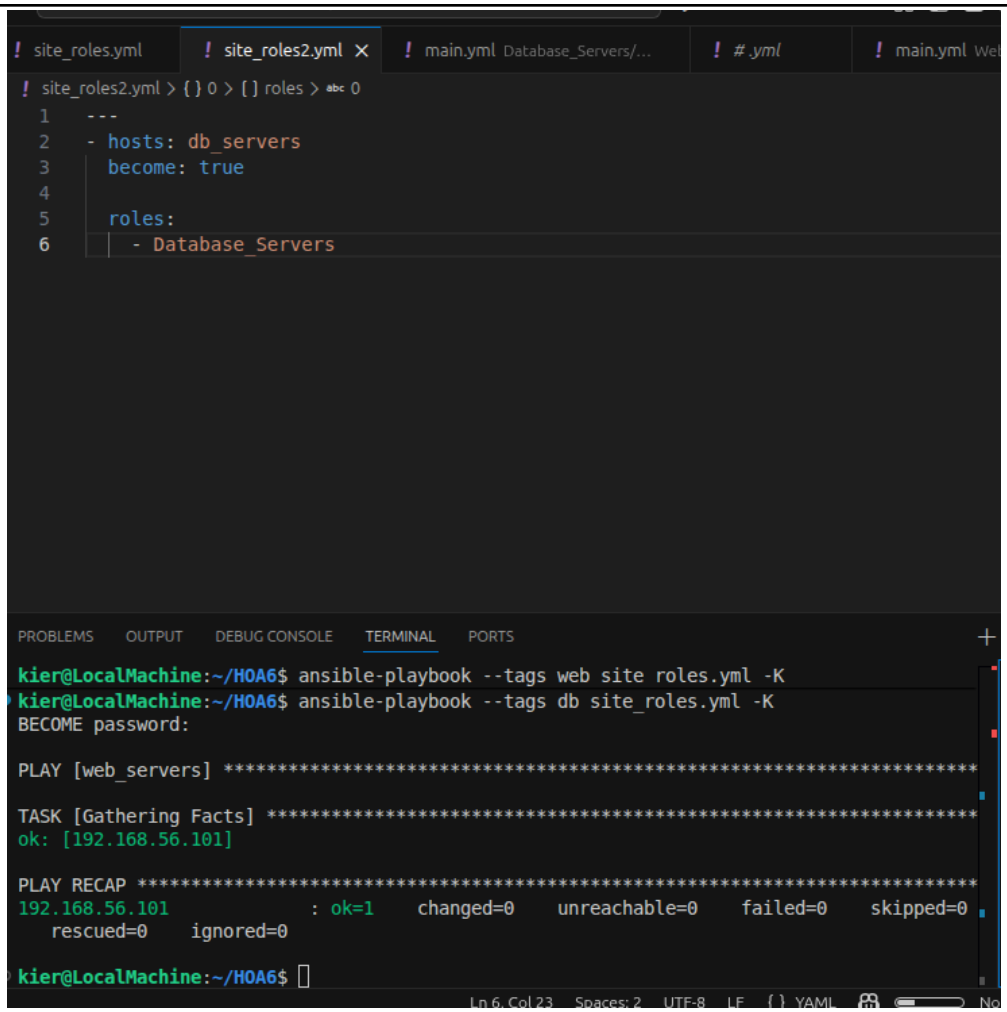
```yaml
---
- hosts: web_servers
  become: true

  roles:
    - Web_Servers
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
kier@LocalMachine:~/HOA6$ ansible-playbook --tags web site roles.yml -K
ok: [192.168.56.101]

TASK [Web_Servers : install apache and php for CentOS servers] *************************
skipping: [192.168.56.101]

TASK [Web_Servers : start httpd (CentOS)] ***********************************************
skipping: [192.168.56.101]

PLAY RECAP ******************************************************************************
192.168.56.101             : ok=2    changed=0    unreachable=0    failed=0    skipped=2
   rescued=0    ignored=0

kier@LocalMachine:~/HOA6$
```

Go to Line/Column

Ln 6, Col 18 Spaces: 2 UTF-8 LF { } YAML

- 

To orchestrate multiple roles in a single playbook for different server groups. Add tags to each role to allow selective execution, e.g., tags: web for webserver, tags: db for database.

Reflections:

Answer the following:

1. What is the importance of putting our remote servers into groups?
   - Grouping servers lets you target specific machines (like web or database servers) so you don't run the wrong tasks on the wrong servers.
2. What is the importance of tags in playbooks?
   - Tags let you run only certain parts of a playbook, which saves time and reduces risk when testing or updating.
3. Why do think some services need to be managed automatically in playbooks?
   - Managing services automatically ensures they stay running, restart after updates, and stay consistent across all servers.